LAPORAN PRAKTIKUM

Internet of Think II



Oleh :

Nama : Nabil Hasani

NPM : 25782077

Kelas : TRI (3C)

Dosen Pengampu :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.pd.,M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG TAHUN 2026

Modul 7: Integrasi Backend Server Lokal (Docker & Mosquitto) dan Rule Engine (Node-RED)

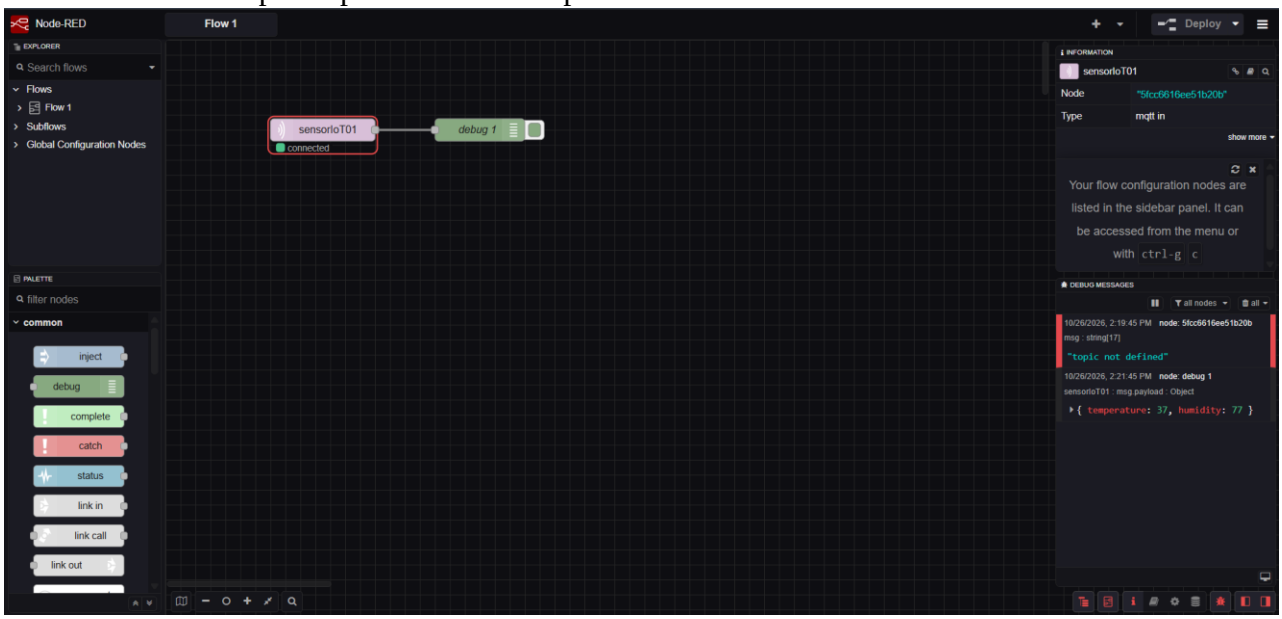
7.5 Praktikum

7.5.1 Praktikum 1: Pembangunan Ekosistem IoT Server dengan Docker Compose

Pada praktikum ini, kita akan membangun infrastruktur server IoT lokal terpadu (Mosquitto MQTT Broker dan Node-RED) secara otomatis hanya menggunakan berkas deklaratif Docker Compose.

Langkah Kerja:

1. Buka File Explorer di laptop Anda, buat direktori kerja baru bernama: **iot-server** (misalnya pada direktori D:/iot-server atau C:/Users/NamaAnda/iot-server).
2. Di dalam folder iot-server, buat struktur sub-direktori berikut secara manual
3. Buka teks editor (VS Code atau Notepad), buat berkas baru bernama mosquitto.conf, simpan di dalam folder iot-server/mosquitto/config/, lalu masukkan konfigurasi parameter berikut:
4. Buka teks editor (VS Code atau Notepad), buat berkas baru bernama mosquitto.conf, simpan di dalam folder iot-server/mosquitto/config/, lalu masukkan konfigurasi parameter berikut:
5. Jalankan **Terminal** (Linux/macOS) atau **PowerShell / Command Prompt** (Windows), arahkan direktori aktif ke dalam folder iot-server:
6. Eksekusi perintah berikut untuk mengunduh (*pull*) *image* dan menjalankan kontainer di latar belakang (*detached mode*):
7. Tunggu hingga terminal menampilkan status [+] Running 2/2 dengan keterangan hijau Started.
8. Verifikasi bahwa kedua kontainer telah aktif dan berjalan normal dengan mengecek daftar proses kontainer:
9. Buka peramban web pada laptop Anda dan ketikkan alamat URL: http://localhost:1880. Jika antarmuka visual kanvas Node-RED berhasil tampil di layar, maka fondasi server IoT lokal Anda telah siap beroperasi secara sempurna!



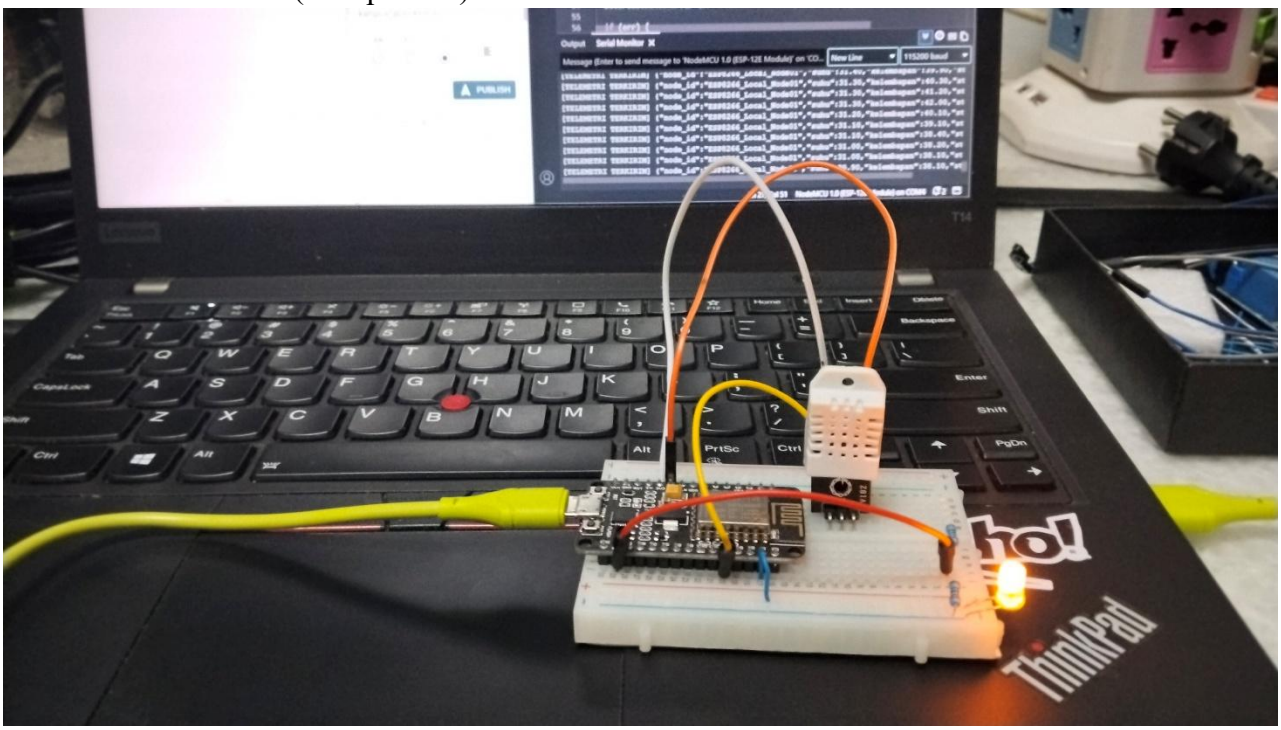
7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Setelah server lokal berjalan di laptop, langkah berikutnya adalah mengarahkan NodeMCU ESP8266 dari broker publik ke alamat IP lokal laptop Anda dalam jaringan Wi-Fi lokal.

Skema Rangkaian Perangkat Keras:

Rangkaian perangkat keras pada praktikum ini identik dengan konfigurasi pada Modul 6 (hanya menggunakan sensor DHT dan indikator LED):

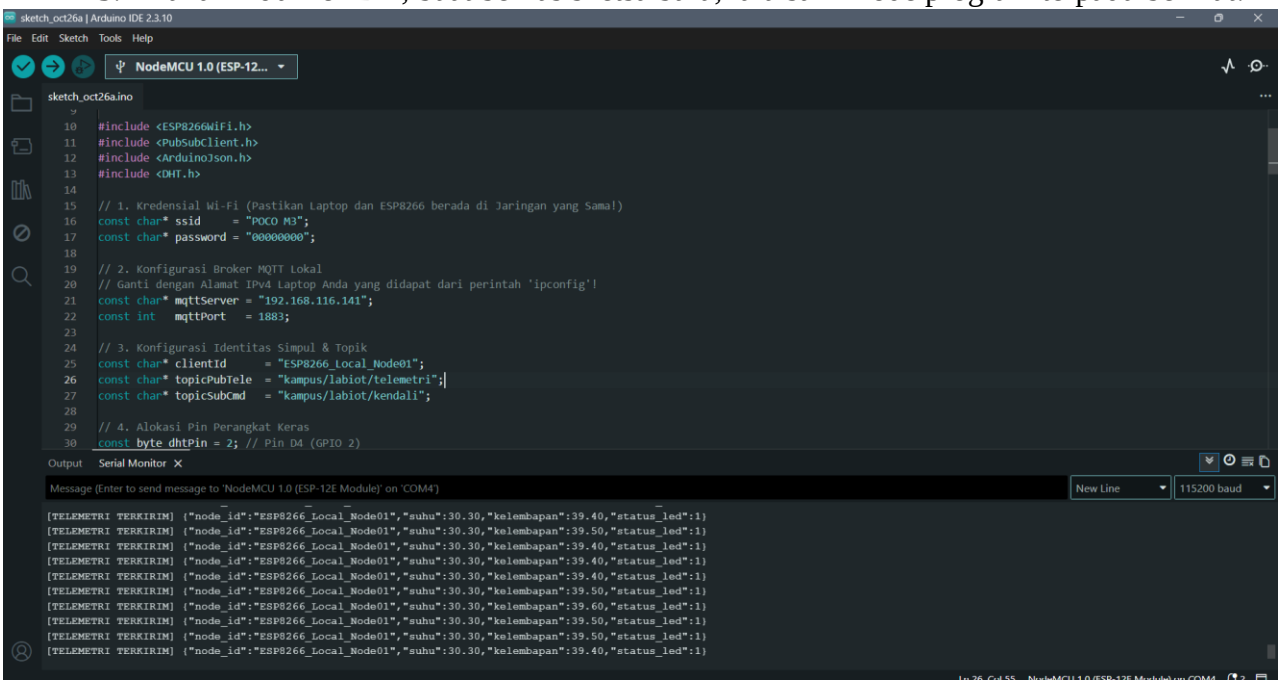
- **Sensor DHT11/DHT22:**
  - Pin VCC → Pin 3V3 NodeMCU
  - Pin GND → Pin GND NodeMCU
  - Pin Data → Pin **D4 (GPIO 2)**
- **LED Indikator Aktuator:**
  - Anoda (kaki panjang) → Pin **D1 (GPIO 5)**
  - Katoda (kaki pendek) → Resistor 220 Ω → Pin GND



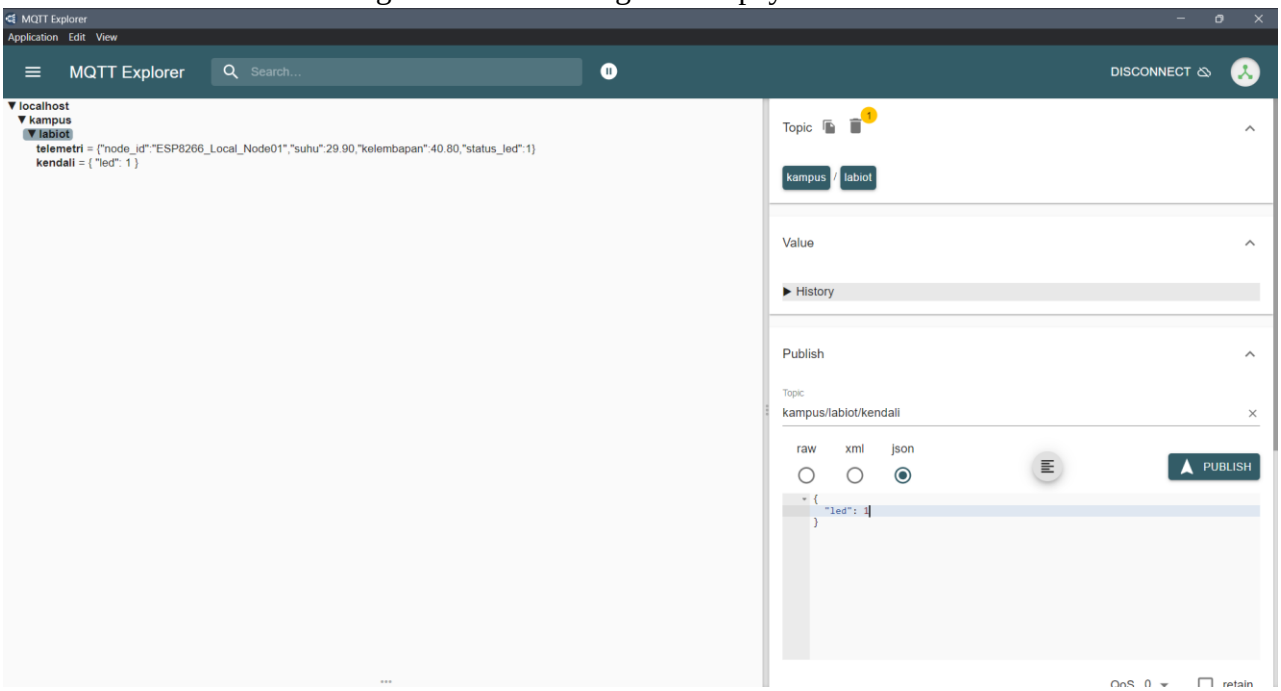
Langkah Pemrograman:

1. Cari tahu Alamat IPv4 Lokal laptop Anda yang terhubung ke jaringan Wi-Fi:
Windows: Buka CMD / PowerShell, ketik ipconfig. Temukan bagian Wireless LAN adapter Wi-Fi, lalu catat baris IPv4 Address (misalnya: 192.168.1.15).
macOS / Linux: Buka terminal, ketik ifconfig atau ip a (misalnya: 192.168.1.15).
2. **[PERINGATAN PENTING FIREWALL WINDOWS]:**
   Sistem keamanan *Windows Defender Firewall* sering kali secara otomatis memblokir koneksi masuk (*inbound connection*) dari port 1883. Jika NodeMCU gagal terhubung ke laptop, buka **Windows Defender Firewall with Advanced Security**, klik **Inbound Rules > New Rule**, pilih **Port > TCP**, masukkan port 1883, lalu pilih **Allow the connection**.

3. Buka **Arduino IDE**, buat berkas sketsa baru, lalu salin kode program terpadu berikut:



4. Unggah (Upload) kode program ke board NodeMCU ESP8266.
5. Buka Serial Monitor pada kecepatan 115200 baud. Pastikan mikrokontroler berhasil mendapatkan alamat IP dan memunculkan tulisan [MQTT] Menghubungkan ke Broker Lokal... Terhubung! serta mulai mengirimkan payload telemetri JSON secara stabil.



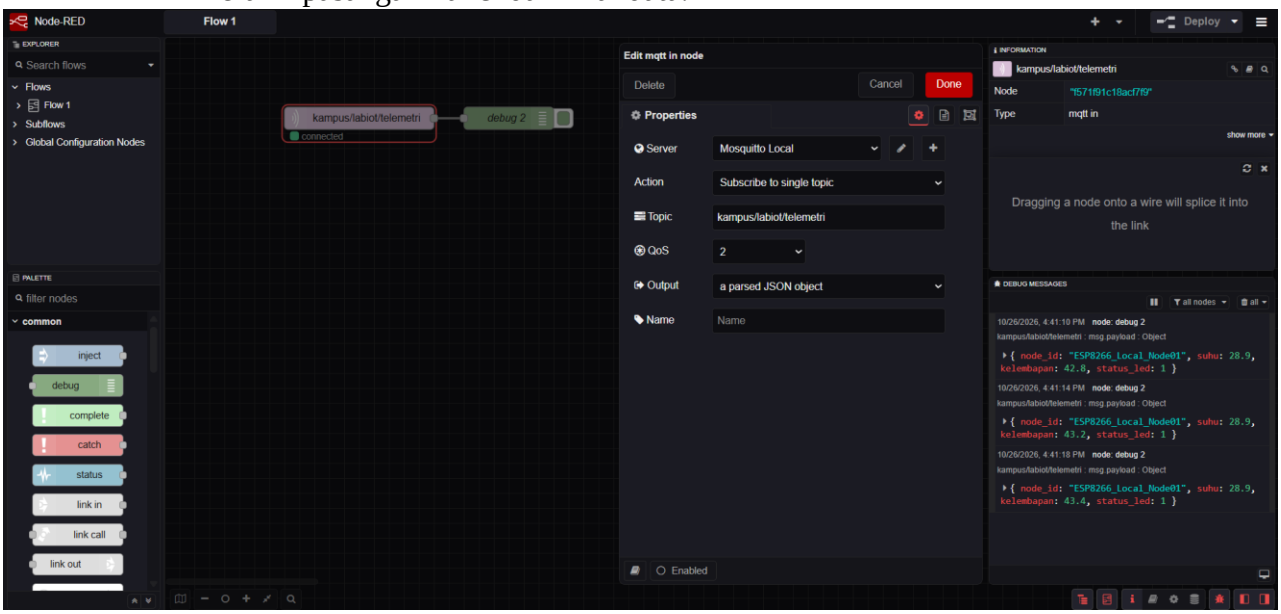
7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

Pada praktikum ini, kita akan menangkap dan membedah aliran data telemetri yang dikirimkan oleh NodeMCU menggunakan antarmuka kanvas Node-RED.

Langkah Kerja:

1. Buka peramban web dan akses antarmuka Node-RED di **http://localhost:1880**.
2. **Menambahkan Konfigurasi Server MQTT Broker:**
    - Di sebelah kiri (*Palette*), temukan kategori **network**, lalu klik dan seret (*drag*) node **mqtt in** ke tengah kanvas kerja.
    - Klik ganda (*double-click*) pada node mqtt in tersebut untuk membuka panel pengaturannya:
        - Pada baris **Server**, klik ikon pensil ( ) untuk membuat konfigurasi broker baru.
        - **Name:** Mosquitto Local
        - **Server:** Ketik **mqtt**
          (PENTING: Jangan gunakan localhost atau IP laptop! Karena Node-RED dan Mosquitto berada dalam satu jaringan Docker Compose yang sama, kontainer Node-RED dapat memanggil broker secara langsung menggunakan nama servis deklaratifnya, yaitu mqtt).
        - **Port:** 1883
        - Klik tombol merah **Add** di pojok kanan atas dialog.
3. **Mengonfigurasi Parameter Topik:**
    - Pada baris **Topic**, masukkan: kampus/labiot/telemetri
    - Pada baris **Output**, ubah dari *auto-detect* menjadi **a parsed JSON object**. (Fitur ini secara otomatis mengurai teks string JSON menjadi objek JavaScript hierarkis tanpa perlu menulis fungsi *parser* manual).
    - Klik tombol **Done**.
4. **Menambahkan Node Pemantau (*Debug Node*):**
    - Pada *Palette* sebelah kiri di kategori **common**, seret node **debug** ke kanvas.
    - Tarik garis (*wire*) dari titik port keluaran di sisi kanan node mqtt in ke port masukan di sisi kiri node debug.
5. **Menerapkan Alur Kerja (*Deploy*):**
    - Klik tombol merah **Deploy** di sudut kanan atas antarmuka.
    - Amati bahwa di bawah node mqtt in muncul teks indikator hijau bertuliskan **connected**.

6. **Menganalisis Aliran Data:**
    - Klik tab ikon kumbang (🐞) di bilah sisi kanan (*Sidebar*) untuk membuka panel **Debug Messages**.
    - Anda akan melihat objek JSON masuk setiap 4 detik secara *real-time*
    - Klik tanda panah segitiga kecil di samping teks Object untuk mengeksplorasi hierarki pasangan kunci dan nilai data.



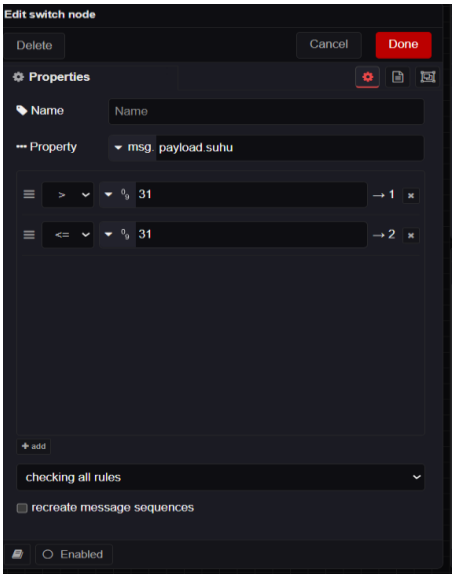
7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

Pada praktikum ini, kita akan membangun *Closed-Loop Rule Engine* otonom:

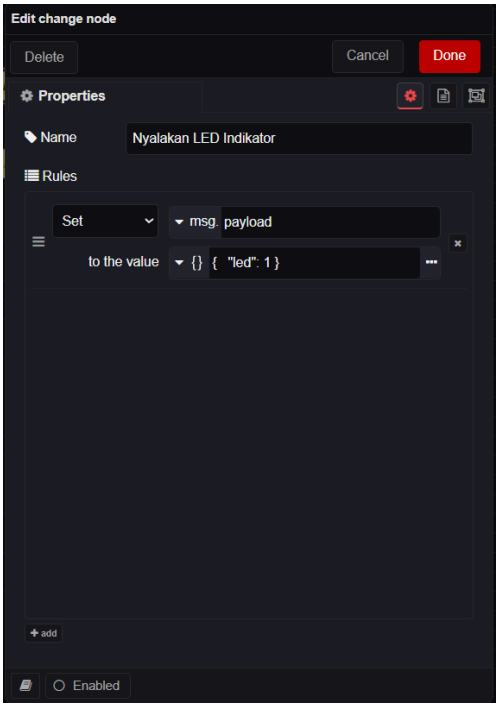
- **JIKA** nilai suhu yang dibaca oleh sensor melebihi ambang batas kritis **31.0∘C**: Node-RED secara otomatis memublikasikan paket JSON {"led": 1} untuk menyalakan LED indikator peringatan bahaya di pin D1.
- **JIKA** suhu berada dalam batas normal (≤31.0∘C): Node-RED memublikasikan paket JSON {"led": 0} untuk memadamkan LED indikator.

Langkah Kerja:

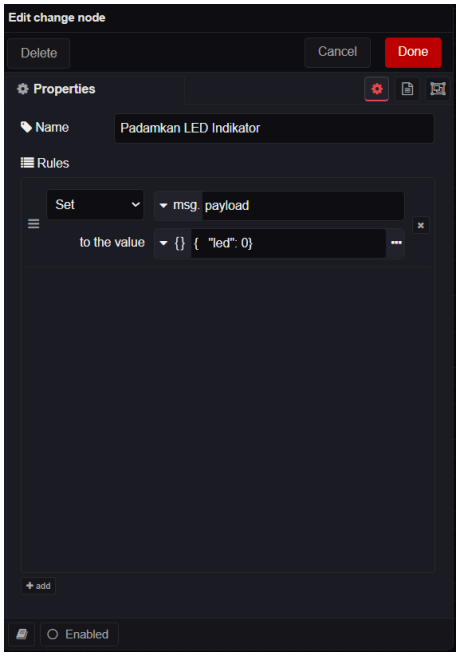
1. **Membuat Node Percabangan Logika (switch):**
    - Pada kategori **function**, seret node **switch** ke kanvas.
    - Hubungkan port keluaran node mqtt in ke port masukan node switch.
    - Klik ganda node switch untuk menyetel aturan logika evaluasi:
        - **Property:** Ganti msg.payload menjadi **msg.payload.suhu**.
        - **Aturan 1 (Kondisi Panas):** Pilih operator >, lalu masukkan angka: **31**.
        - Klik tombol **+ add** di bagian bawah untuk menambahkan aturan kedua.
        - **Aturan 2 (Kondisi Aman):** Pilih operator <=, lalu masukkan angka: **31**.
        - Klik **Done**. Perhatikan bahwa node switch kini memiliki dua port keluaran bernomor 1 (atas) dan 2 (bawah).



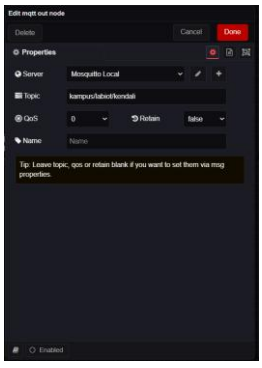
2. **Membuat Node Transformasi Muatan Panas (change):**
    - Seret node **change** dari kategori *function* ke kanvas di sebelah kanan atas node switch.
    - Hubungkan port keluaran 1 (suhu >31) dari node switch ke masukan node change pertama ini.
    - Klik ganda node change:
        - **Name:** Nyalakan LED Indikator
        - **Set:** msg.payload
        - **To:** Ubah tipe data dari string (az) menjadi **JSON** ({ }), lalu masukkan nilai:
        - {"led": 1}
        - Klik **Done**.



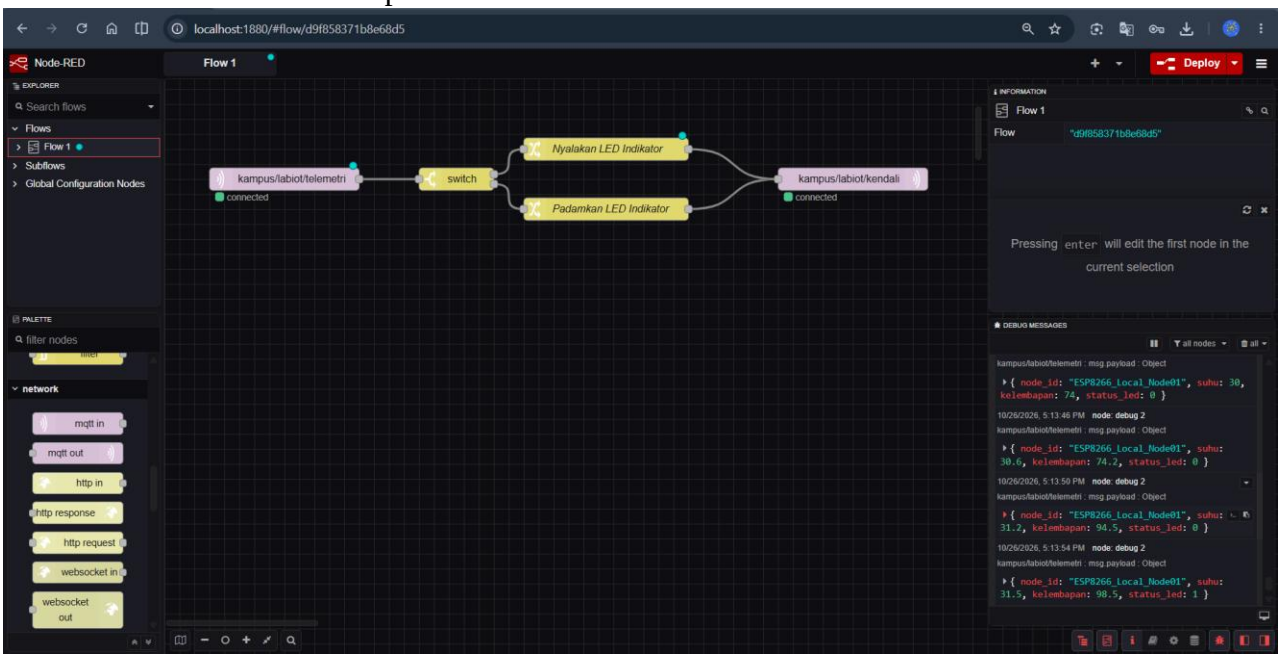
3. **Membuat Node Transformasi Muatan Normal (change):**
    - Seret node **change** kedua ke kanvas di sebelah kanan bawah node switch.
    - Hubungkan port keluaran 2 (suhu ≤31) dari node switch ke masukan node change kedua ini.
    - Klik ganda node change:
        - **Name:** Padamkan LED Indikator
        - **Set:** msg.payload
        - **To:** Ubah tipe data menjadi **JSON** ({ }), lalu masukkan nilai:
        - {"led": 0}
        - Klik **Done**.



4. **Menerbitkan Instruksi Balik ke Broker (mqtt out):**
    - Pada kategori **network**, seret node **mqtt out** ke kanvas.
    - Hubungkan keluaran dari node change pertama dan node change kedua secara bersamaan ke port masukan node mqtt out.
    - Klik ganda node mqtt out:
        - **Server:** Pilih Mosquitto Local.
        - **Topic:** kampus/labiot/kendali
        - **QoS:** 0 (atau 1)
        - **Retain:** false
        - Klik **Done**.



5. **Menerapkan Perubahan (*Deploy*):**
    - Klik tombol merah **Deploy** di kanan atas antarmuka.
6. **Pengujian Fisik Sistem Otomasi Tertutup:**
    - Amati status awal: Karena suhu ruangan umumnya berkisar antara 26∘C−29∘C (≤31∘C), LED indikator berada dalam kondisi padam.
    - Dekatkan sumber panas ringan (seperti ujung jari Anda yang ditempelkan kuat pada sensor DHT, atau hembusan napas hangat) hingga pembacaan suhu pada panel Debug merangkak naik melewati 31.0∘C.
    - **Hasil Pengamatan:** Seketika suhu melampaui 31.0∘C, Node-RED mengeksekusi port 1, mengirimkan payload {"led": 1}, dan LED indikator bahaya di pin D1 seketika menyala terang!
    - Lepaskan jari Anda dan biarkan suhu mendingin kembali ke bawah 31.0∘C. LED indikator akan padam kembali secara otomatis.



7.6 Latihan

Kerjakan latihan berikut secara mandiri berbasis analisis teori dan eksperimen teknis untuk melengkapi laporan praktikum Anda:

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):**
  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks **mqtt**, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.
    i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?

  Karena localhost mengarah ke container Node-RED sendiri, bukan container Mosquitto. Solusinya, gunakan nama layanan broker, yaitu mqtt, dengan port 1883.

ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!

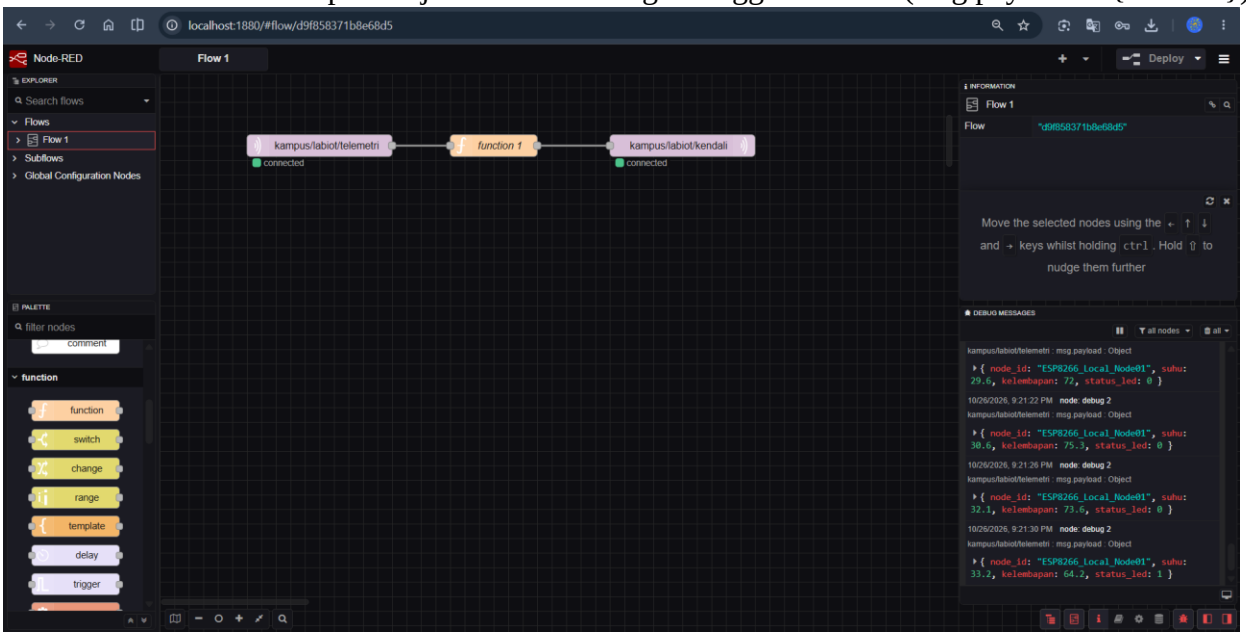
Docker menyediakan DNS internal yang menerjemahkan nama layanan mqtt menjadi alamat IP container Mosquitto. Jaringan internal Docker memungkinkan Node-RED terhubung ke Mosquitto tanpa perlu mengatur IP container secara manual.

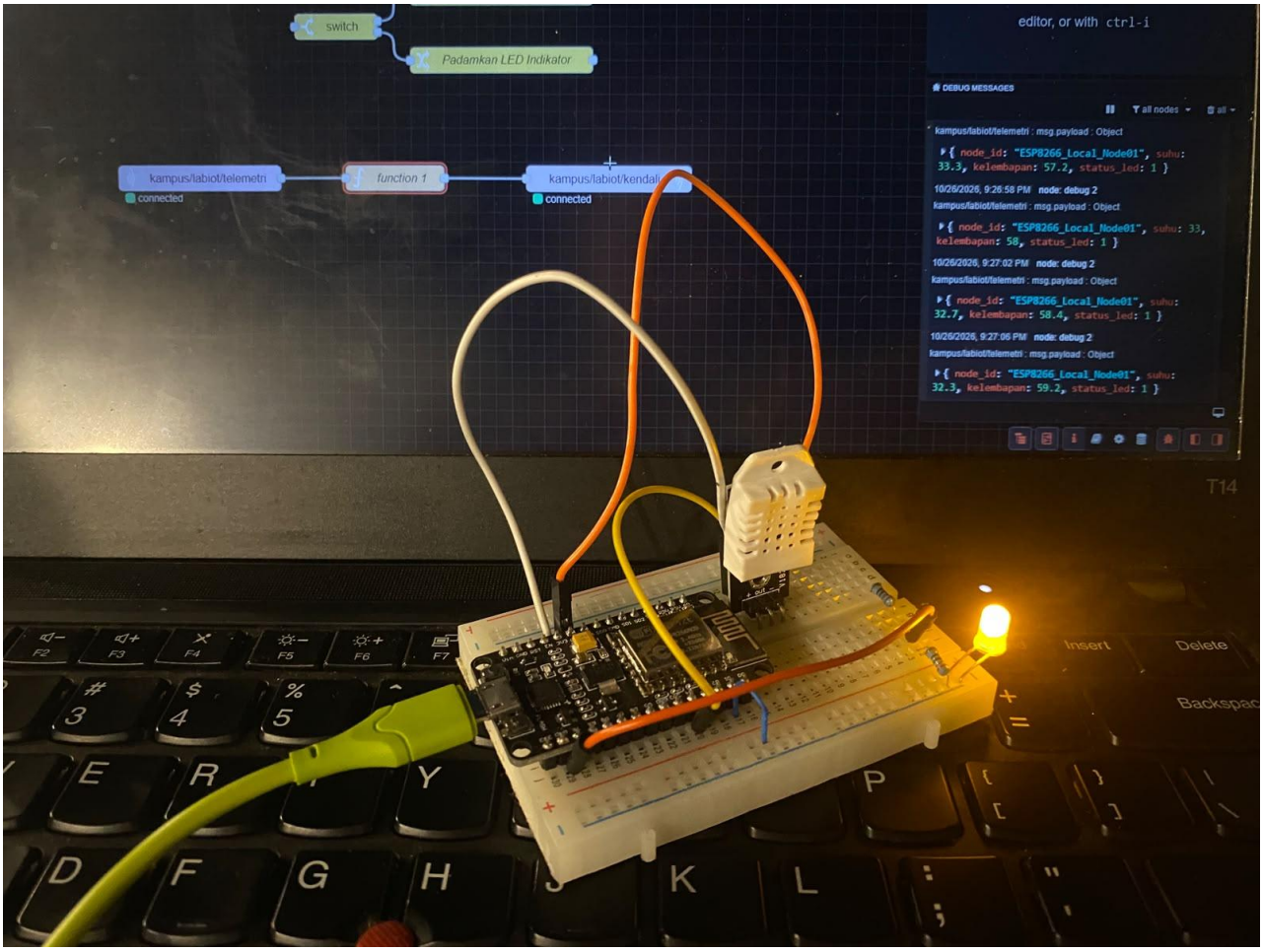
- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):**
Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.
  i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?

  NodeMCU memiliki IP 192.168.43.102, sedangkan laptop menggunakan IP 192.168.1.15. Keduanya berada pada subnet berbeda sehingga tidak dapat berkomunikasi secara langsung tanpa routing.

  ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

  Hubungkan NodeMCU dan laptop ke jaringan yang sama, gunakan IP laptop sebagai alamat broker MQTT, dan pastikan firewall mengizinkan koneksi melalui port 1883.
- **Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):**
Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0∘C (misal: 31.05∘C→30.95∘C→31.02∘C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:
  - LED baru menyala jika suhu >31.5∘C (msg.payload = {"led": 1}).
  - LED baru padam jika suhu mendingin hingga <30.0∘C (msg.payload = {"led": 0}).





Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

```
let temp = msg.payload.suhu;
let currentState = context.get('ledState') || 0;

// Logika Histeresis
if (temp > 31.5) {
    currentState = 1; // Nyalakan LED jika suhu melebihi batas atas kritis
} else if (temp < 30.0) {
    currentState = 0; // Padamkan LED jika suhu mendingin di bawah batas bawah
aman
}

context.set('ledState', currentState);
msg.payload = { "led": currentState };
return msg;
```

- **Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):**
  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?

Menghapus volume mapping dapat menyebabkan data yang tersimpan di dalam container tidak lagi terhubung dengan folder penyimpanan di host. Jika container dibuat ulang, data yang hanya tersimpan di dalam container berisiko hilang.

ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!

Bind Mount: Menghubungkan folder atau file tertentu di komputer host ke dalam container, contohnya ./nodered:/data.

Named Volume: Menggunakan volume yang dikelola oleh Docker untuk menyimpan data, contohnya nodered_data:/data.

Keduanya digunakan untuk mempertahankan data agar tidak hanya tersimpan di dalam container.

7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

Dalam dunia industri nyata, membiarkan MQTT Broker beroperasi tanpa kata sandi (allow_anonymous true) adalah pelanggaran fatal terhadap standar keamanan siber (*Cybersecurity Vulnerability*). Pada tugas ini, kembangkan ekosistem server IoT lokal Anda menjadi sistem yang terproteksi penuh menggunakan sistem autentikasi kredensial (*Username & Password*).

- **Tujuan:** Mampu mengonfigurasi berkas kata sandi terenkripsi di dalam kontainer Mosquitto, menonaktifkan akses anonim, menganalisis dampak penolakan koneksi (*rc=5*), serta memperbarui konfigurasi klien ESP8266 dan Node-RED secara mandiri.
- **Tingkat Kesulitan:** Lanjut (Mandiri / Individu)

Instruksi Pengerjaan Langkah demi Langkah:

1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:

Buka terminal/PowerShell di laptop Anda, lalu jalankan perintah berikut untuk mengeksekusi utilitas mosquitto_passwd langsung di dalam kontainer Mosquitto yang sedang berjalan:

docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin

- Sistem akan meminta Anda memasukkan kata sandi. Masukkan kata sandi: **rahasia123** (teks tidak akan muncul di layar saat diketik), lalu tekan Enter dan konfirmasi ulang kata sandi tersebut.
- Buka folder iot-server/mosquitto/config/ pada laptop Anda, pastikan kini terdapat berkas baru bernama password.txt yang berisi baris *hash* kredensial akun admin.

2. Memperbarui Konfigurasi Kebijakan Mosquitto:

Buka berkas iot-server/mosquitto/config/mosquitto.conf di editor teks Anda:

- Ubah nilai allow_anonymous true menjadi:

allow_anonymous false

- Tambahkan baris pemanggilan berkas kata sandi:

password_file /mosquitto/config/password.txt

- Simpan berkas mosquitto.conf.

3. Memuat Ulang Layanan Broker:

Restart kontainer mqtt-broker melalui terminal agar kebijakan keamanan baru diterapkan:
docker restart mqtt-broker

4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):

- Buka antarmuka Node-RED di browser: Amati bahwa di bawah node mqtt in dan mqtt out, indikator status kini berubah menjadi warna merah bertuliskan **disconnected** atau **connection refused**.
- Buka Serial Monitor NodeMCU: Amati bahwa koneksi mikrokontroler kini gagal dan berulang kali menampilkan kode penolakan:
- [MQTT] Menghubungkan ke Broker Lokal... Gagal (rc=5). Coba lagi dalam 4 detik...

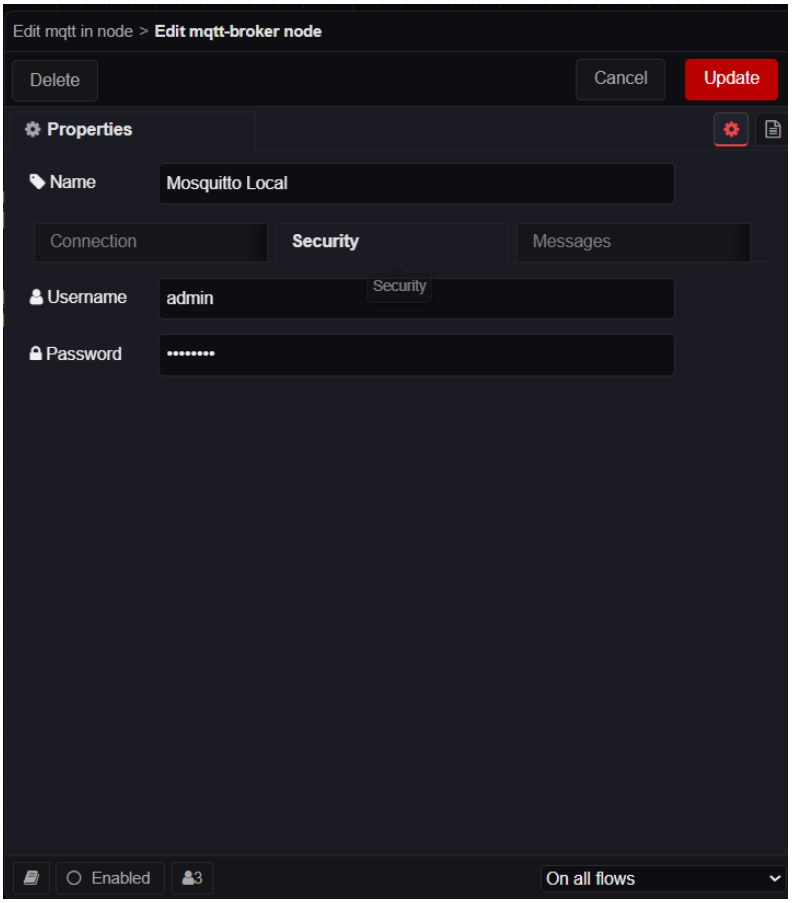
(Catatan: Kode status rc=5 merupakan representasi standar MQTT untuk 'Connection Refused: Not Authorized').

5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

1. **Pemutakhiran Skrip Program NodeMCU ESP8266:**
    - Buka sketsa program Arduino IDE dari Praktikum 2.
    - Modifikasi bagian pemanggilan fungsi client.connect() di dalam fungsi reconnectMQTT() dengan menyertakan kredensial resmi:
    - // Format: client.connect(clientId, username, password)
    - if (client.connect(clientId, "admin", "rahasia123")) {
    - Serial.println(" Terhubung dengan Autentikasi Sah!");
    - client.subscribe(topicSubCmd);

}

    - Unggah ulang kode ke NodeMCU dan amati bahwa ESP8266 kini kembali berhasil terhubung (*rc=0*).
2. **Pemutakhiran Konfigurasi Kredensial pada Node-RED:**
    - Pada kanvas Node-RED, klik ganda node mqtt in.
    - Klik ikon pensil ( ) di samping nama broker Mosquitto Local.
    - Buka tab **Security** di bagian atas dialog konfigurasi.
    - Masukkan **Username:** admin dan **Password:** rahasia123.
    - Klik tombol **Update**, lalu klik **Done**, dan tekan tombol merah **Deploy**.
    - Amati bahwa indikator node MQTT pada kanvas Node-RED seketika kembali berwarna hijau (**connected**), dan aliran data telemetri serta kendali otomasi indikator LED loop tertutup kembali berfungsi normal dan aman dari gangguan pihak luar!



Kriteria Keberhasilan:

- Broker Mosquitto terbukti menolak akses anonim tanpa kredensial (allow_anonymous false).

```
PS C:\docker\IoT-server3> docker run --rm eclipse-mosquitto:latest mosquitto
_sub -h host.docker.internal -p 1883 -t "uji/anonim" -W 5 -d
Client null sending CONNECT
Connection error: Connection Refused: not authorised
Client (null) received CONNACK (5)
Client null sending DISCONNECT

What's next:
    Debug this container error with Gordon → docker ai "help me fix this con
tainer error"
PS C:\docker\IoT-server3>
```

```
listener 1883
protocol mqtt

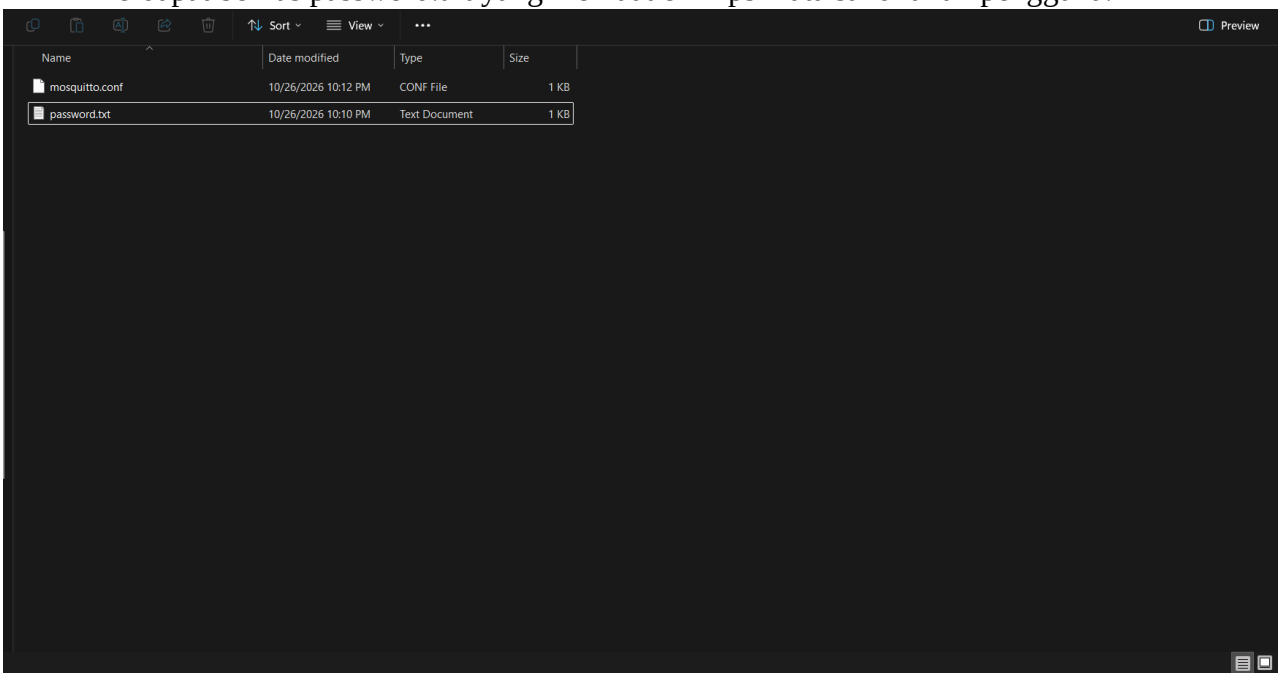
listener 9001
protocol websockets

allow_anonymous false
password_file /mosquitto/config/password.txt

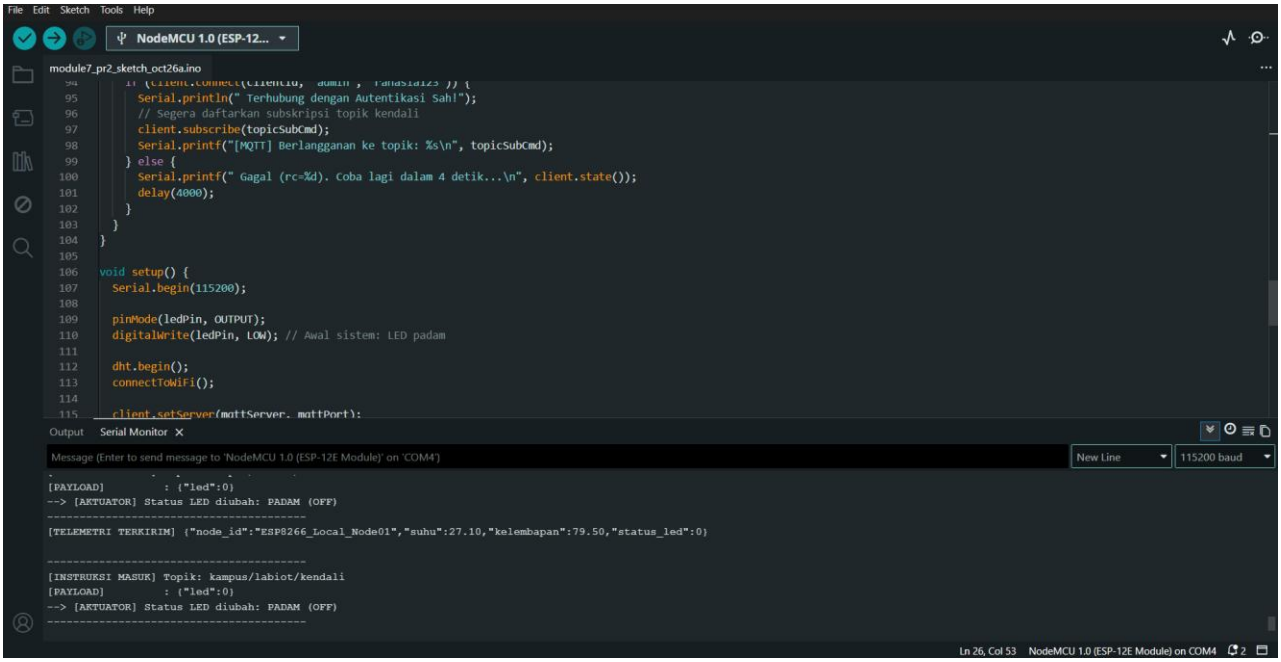
persistence true
persistence_location /mosquitto/data/

log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

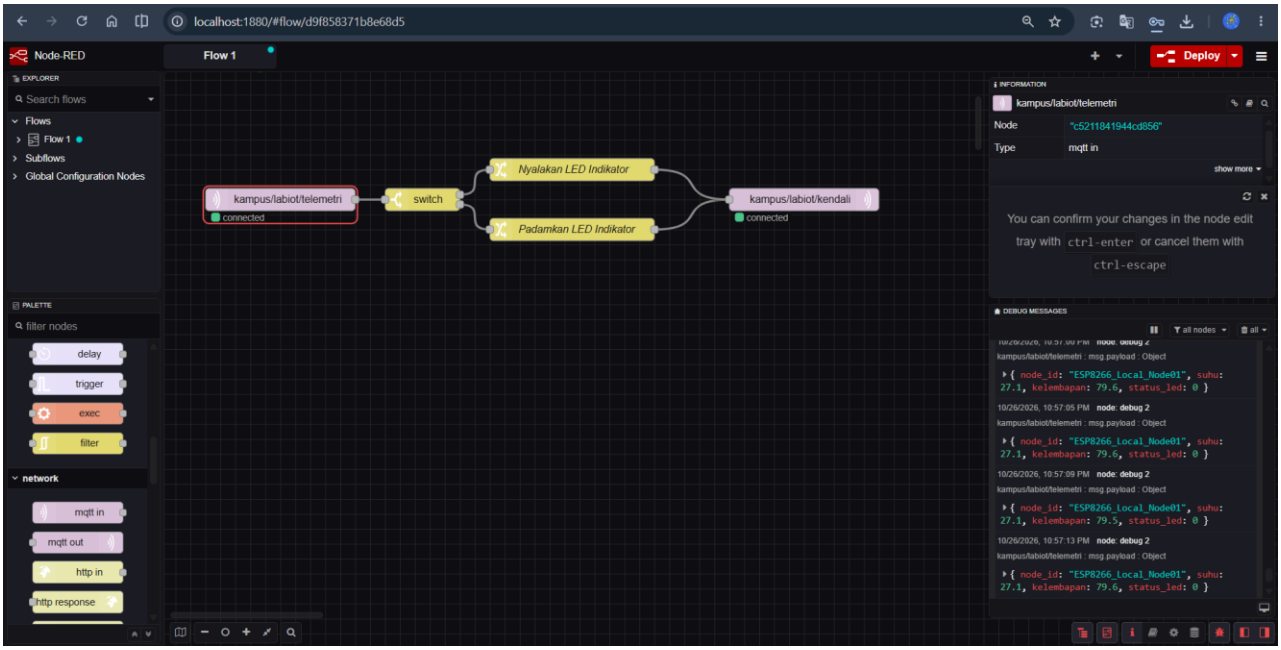
- Terdapat berkas password.txt yang memuat enkripsi kata sandi akun pengguna.



- Serial Monitor NodeMCU membuktikan pemulihan koneksi sukses menggunakan kredensial autentikasi resmi.



- Seluruh aliran kerja (*flow*) pada Node-RED kembali terhubung dan mampu memicu otomasi LED secara stabil.


localhost:1880/#flow/d9f858371b8e68d5
Node-RED
Flow 1
Deploy
EXPLORER
Search flows
Flows
Flow 1
Subflows
Global Configuration Nodes
kampus/labiot/telemetri
connected
switch
Nyalakan LED Indikator
Padamkan LED Indikator
kampus/labiot/kendali
connected
INFORMATION
kampus/labiot/telemetri
Node
"c5211841944cd856"
Type
mqtt in
show more
You can confirm your changes in the node edit tray with ctrl-enter or cancel them with ctrl-escape
PALETTE
filter nodes
delay
trigger
exec
filter
network
mqtt in
mqtt out
http in
http response
DEBUG MESSAGES
all nodes
all
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.1, kelembapan: 79.6, status_led: 0 }
10/26/2026, 10:57:05 PM node: debug 2
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.1, kelembapan: 79.6, status_led: 0 }
10/26/2026, 10:57:09 PM node: debug 2
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.1, kelembapan: 79.5, status_led: 0 }
10/26/2026, 10:57:13 PM node: debug 2
kampus/labiot/telemetri : msg.payload : Object
{ node_id: "ESP8266_Local_Node01", suhu: 27.1, kelembapan: 79.6, status_led: 0 }